



Aegis

AI Defence Lab for Payment Security

Closed-loop red team / blue team system — identify → generate → defend

A complete technical explanation of the project: what it does, how every part works, every term defined, the full technology stack, how to run and test it, and the measured results with their limitations stated.

Competition	Mastercard Innovation Challenge 2026 — AI Defense Lab for Payment Security
Organiser	Mastercard AI Garage
Entry	Solo
Repository	Deven-Kulthia/ThreatForge (public)
Backend language	Python 3.14.4
Frontend language	TypeScript 5.7 (React 19)
Code size	4,210 lines backend · 2,176 lines frontend · 857 lines tests
Tests	113 passing, plus 6 module self-checks
Metrics generated	2026-08-17T19:03:48Z
Data	100% synthetic — no real cardholder data, PII or production payment data

Every figure in this document is read from `artifacts/metrics.json`, produced by `backend.app.evaluate`. No number is typed by hand.

Contents

1. The problem, in plain terms
2. What Aegis is — red team, blue team, and the loop
3. Technology stack — every language, library and tool, and why
4. Architecture — module by module
5. The data model — every field explained
6. Pillar 1 — Identify: the 25 attack vectors
7. Pillar 2 — Generate: simulation and its fidelity evidence
8. Pillar 3 — Defend: features, cascade, explainability
9. Evaluation — methodology and full results
10. Where it fails — stated plainly
11. The dashboard — all seven panels
12. How to run it
13. How to test it
14. Glossary — every term used
15. Competition compliance
16. Repository map

1 · The problem, in plain terms

A bank's fraud detector is like a security guard who has been shown photographs of every criminal caught so far. He is excellent at recognising those faces. A new criminal — a face in no photograph — walks straight past him. He is not incompetent; he simply has no photograph.

That is supervised fraud detection. It learns from fraud that **already happened**, and it learns slowly, because the labels come from chargebacks: customers disputing charges, which takes **weeks**.

Generative AI changed the other side of that equation. Inventing a plausible new fraud technique became cheap and fast:

- Large language models sustain thousands of individually tailored scam conversations at once.
- Generated documents and deepfake video defeat identity checks at account opening.
- Autonomous agents now transact on a cardholder's behalf, blurring what "the customer did".
- Iteration is cheap: probe, observe the decline, adapt, retry.

The asymmetry. Attackers invent new techniques in days. Defenders receive the labels needed to learn them in weeks. A novel typology is, by definition, out of distribution — you cannot classify your way out of it with a better model.

So Aegis inverts the order: **generate the unseen fraud first**, then train and stress-test the defence on it, under constraints that keep the generated fraud realistic.

2 · What Aegis is — red team, blue team, and the loop

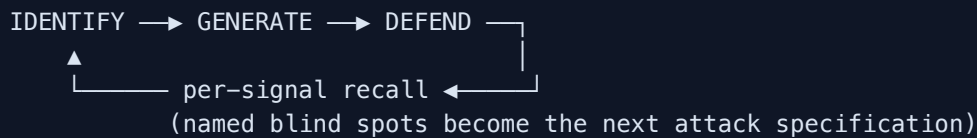
The competition brief frames this as a **red team / blue team** challenge in which entrants "take on both sides of the problem". Both sides are first-class in Aegis, and the dashboard labels them as such.

Side	Job	Where it lives
Red team	Research and generate novel attacks; declare, before running, which detection signals each attack <i>should</i> trip	Attack Simulator panel · <code>attacks.py</code>
Blue team	Detect, score, explain, and report back which of those declared signals were missed	Live Stream, Investigate, Fraud Network, Performance · <code>detect.py</code> , <code>explain.py</code> , <code>evaluate.py</code>

The three pillars

#	Pillar	What it does
1	Identify	Map the emerging GenAI-powered threat surface — 25 vectors across 10 categories, each mapped to MITRE ATLAS
2	Generate	Simulate those attacks at scale with high fidelity — 90,258 transactions, 75 campaigns
3	Defend	Detect them with a calibrated, explainable cascade — PR-AUC 0.944, p99 18.8 ms

Why it is a loop and not a pipeline



Every generated attack emits an `expected_signals` list **before** it executes. Detection is then graded against that declaration, per signal, rather than only per transaction. That converts a vague "we caught 89% of fraud" into "here are the exact mechanisms we are blind to" — which is actionable, and becomes the specification for the next round of attacks.

Coverage. 39 of 39 distinct declared signals are implemented. The 8 that are not are **named with reasons** (see §8), because a named gap is engineering judgement whereas an unexplained gap looks like a defect.

3 · Technology stack — every language, library and tool

Languages

Language	Version	Used for	Lines
Python	3.14.4	All backend: data generation, attack simulation, feature engineering, detection, explainability, evaluation, API	4,210
TypeScript	5.7	The entire dashboard, fully type-checked	2,176
SQL	SQLite dialect	Append-only audit trail	inline
Bash	—	<code>verify.sh</code> one-command verification gate	~150
HTML / CSS	—	Tailwind utility classes; this PDF's own template	—

Python libraries (runtime)

Library	Version	Licence	Why this one
numpy	2.5.2	BSD-3	Vectorised numerics; the generator builds whole populations as arrays rather than row loops
pandas	3.0.5	BSD-3	The transaction table and all group-wise feature aggregation
scikit-learn	1.9.0	BSD-3	<code>HistGradientBoostingClassifier</code> for the model stage, <code>IsotonicRegression</code> for calibration, metrics
networkx	3.6.1	BSD-3	Entity graph: connected components, fan-in, shared-infrastructure detection
fastapi	0.141.1	MIT	REST + WebSocket API with automatic schema
uvicorn	0.52.3	BSD-3	ASGI server
sqlite3	stdlib	Public domain	Audit trail — zero-setup, no service to run, and it is already in the standard library

Python libraries (tooling, not on the decision path)

Library	Version	Licence	Why
pytest	9.1.1	MIT	113 tests across 4 suites
python-pptx	1.0.2	MIT	Generates the walkthrough deck from <code>metrics.json</code> so slides cannot drift from code
playwright	—	Apache-2.0	Real-browser smoke test; also prints this PDF, avoiding a second document toolchain

Frontend

Package	Version	Licence	Why
react / react-dom	19.0	MIT	UI runtime
vite	6.0	MIT	Dev server and build; fast HMR
tailwindcss	4.0	MIT	Styling without a bespoke design system
recharts	2.15	MIT	Charts — alert bands, calibration curve
lucide-react	0.470	ISC	Icons
typescript	5.7	Apache-2.0	Type safety; <code>tsc --noEmit</code> is part of the verification gate

Runtime: **Node 24.18.0**, **npm 11.16.0**.

Licence compliance. All 41 Python and 121 npm packages were audited: every one is OSI-approved and permissive (MIT / BSD / Apache-2.0 / ISC / MPL-2.0). Zero AGPL, GPL or SSPL. Kaggle Foundational Rules §6c requires open-source code that "in no event limits commercial use".

Deliberate omissions — and why

Not used	Reason
LightGBM / XGBoost	<code>libomp</code> is unavailable in this environment. scikit-learn's <code>HistGradientBoostingClassifier</code> is the same algorithm family with no OpenMP dependency.
SHAP	Requires <code>numba</code> , which fails to build on Python 3.14. Rather than ship an approximate explainer and call it attribution, the arbiter was designed to be <i>additive</i> , making its decomposition exact arithmetic (see §8).
Graph neural network	The GADBench benchmark shows tree ensembles plus neighbour aggregation outperform purpose-built GNNs on tabular graph anomaly detection. A GNN would cost latency and explainability for no measured gain.
SMOTE / synthetic oversampling	Leaks information across the split and assumes a unimodal minority class; our minority is 25 distinct typologies. Used <code>class_weight="balanced"</code> instead.
An LLM on the decision path	A hosted model in an authorization flow is a latency and availability risk, and a non-deterministic decision is not auditable. The LLM narrates; it never makes the block decision.
Docker / Kubernetes / message queues	Nothing in the demo needs them. Two processes and a file-backed database is the whole runtime.

4 · Architecture — module by module

backend/app/	
schema.py	field definitions, MCC and country tables, the observable/ground-truth split
generator.py	synthetic population + legitimate authorization traffic
attacks.py	25 attack simulators, each emitting full ground-truth metadata
features.py	57 strictly causal features
fidelity.py	measures realism and non-separability of the corpus
detect.py	the three-stage cascade + arbiter + calibration
explain.py	exact additive reason codes and counterfactuals
evaluate.py	PR-AUC, operating points, calibration, latency, zero-day, per-signal
api.py	FastAPI REST + WebSocket + SQLite audit trail

Module	Responsibility	Key design decision
schema.py	One source of truth for every field	GROUND_TRUTH_FIELDS is a frozen set excluded from OBSERVABLE_FIELDS , so label leakage is <i>impossible by construction</i> rather than merely discouraged
generator.py	Cardholders, merchants, devices, and legitimate traffic	Fraud is not produced here. Legitimate and adversarial behaviour come from independent code paths, so the detector cannot exploit a shared generation artefact
attacks.py	The 25 simulators	Each may only manipulate attacker-controllable levers (§7)
features.py	Turn raw authorizations into model inputs	Causality verified by test: recomputing features on a truncated prefix must give identical values, so no feature can see the future
detect.py	Score a transaction	The graph stage is gated by a compute budget (riskiest 20%), not a score threshold — a threshold lets cost spike exactly when an attack floods the high-risk band
explain.py	Say why	Exact additive decomposition, verified against the model's own decision function
evaluate.py	Grade everything	Writes artifacts/metrics.json ; every downstream artifact (deck, this PDF, docs) reads from it
api.py	Serve the dashboard	Append-only audit trail; health endpoint reports readiness so the UI never fires a request it expects to fail

5 · The data model — every field explained

The schema is modelled on **ISO 8583**, the international standard for card authorization messages. That matters: it constrains features to things that genuinely exist at authorization time, and it means anything requiring dispute-lifecycle or session data is honestly out of scope.

Group	Field	Meaning
Identity / routing	transaction_id	Unique id
	timestamp	When the authorization occurred
	card_token	Synthetic network token — never a PAN (card number)
	account_id	The cardholder's account
	issuer_country	Country of the issuing bank
Money	amount	Transaction amount
	currency	Currency, derived from cardholder home country
	amount_local	Amount in home currency, for comparability
Acceptance	merchant_id , merchant_name	The shop
	mcc	Merchant Category Code — 4-digit industry classification (e.g. 5999 = miscellaneous retail)
	merchant_country	Where the merchant is
	merchant_age_days	How long the merchant has existed — new merchants carry genuinely elevated risk
	is_recurring	A subscription-style repeat payment
Presentation	channel	ECOM online · POS physical card machine · MOTO mail/telephone order
	entry_mode	How the credential was supplied (chip, contactless, keyed, NETWORK_TOKEN)
	card_present	Was the physical card there? Card-not-present is the higher-risk case
Device	device_id	Device fingerprint
	ip_prefix	/24 only — no full address retained, even synthetic
	user_agent_hash	Hashed browser signature
Verification	avs_result	Address Verification Service — did the billing address match?
	cvv_result	Was the 3-digit security code correct?
	three_ds_status	3-D Secure — the "verify with your bank" step (Verified by Visa / Mastercard Identity Check)
	sca_exemption	Strong Customer Authentication exemption claimed. Under EU PSD2, some low-risk payments may skip the check — attackers game these bands
	network_token_used	Was a token used instead of a raw card number?
	cross_border	Issuer and merchant in different countries

Outcome	<code>auth_response</code>	Approved or declined
Ground truth lab only — never present in a real authorization message	<code>is_fraud</code>	The label
	<code>attack_type</code>	Which of the 25 vectors
	<code>scenario_id</code>	Which campaign instance
	<code>attack_strength</code>	How aggressive the campaign was (0–1)
	<code>synthetic</code>	Always <code>true</code> — a compliance marker on every row

Scale of the corpus: 1,650 tokenised cards · 263 merchants · 44 days · 90,258 transactions · 3,459 fraudulent (3.83%) · 75 campaigns.

6 · Pillar 1 — Identify: the 25 attack vectors

25 vectors across 10 categories. Each is mapped to **MITRE ATLAS** (a public catalogue of adversarial techniques against AI systems), annotated with the specific role generative AI plays, assigned a severity 1–5, and flagged `hard_to_detect` where it was deliberately built to overlap legitimate behaviour. **12 of 25** carry that flag — the taxonomy is not padded with easy wins.

Category	Representative vectors	What GenAI changed
Synthetic identity	Generated-document application farm; history building; bust-out	LLMs mass-produce coherent applicant personas and plausible life-event narratives, so a fake identity survives manual review
Deepfake / KYC	Liveness and KYC defeat at onboarding	Generated video and documents defeat identity checks that assumed forgery was expensive
Account takeover	Credential stuffing; SIM-swap OTP interception; voice-clone call-centre takeover	Voice cloning defeats phone-based identity verification
Scam / social engineering	APP scam via conversational LLM; romance / pig-butcherer	Thousands of simultaneous, individually tailored grooming conversations
Agentic commerce	Agent impersonation; prompt injection via merchant-controlled fields	Autonomous agents transact on a cardholder's behalf, and text fields become an injection surface
Fraud ring	Coordinated multi-card ring; mule fan-out	Cheap orchestration of many synthetic participants
Merchant fraud	Fabricated storefront; refund collusion; transaction laundering / MCC misrepresentation	Generated storefronts, catalogues and reviews are trivial to produce at scale
Adaptive evasion	Velocity evasion; adaptive mimicry; TRA / SCA-exemption threshold gaming	Models learn the victim's own baseline and stay inside it
Enumeration	BIN enumeration bursts	Automated, distributed card testing
First-party fraud	Mandate replay abuse; dispute abuse patterns	Templated, plausible dispute narratives at scale

Alignment with the organiser's stated priorities. Mastercard AI Garage publicly named four priority threats — synthetic identities, deepfake KYC, fake merchant storefronts and AI-enabled scams. All four are covered as first-class categories.

The hardest case, and why it matters

`APP_SCAM_LLM` (authorised push payment scam) is the hardest problem in payments fraud: the **genuine cardholder**, on **their own device**, with **real 3-D Secure authentication**, **willingly** authorises the payment. Every credential signal is clean. Only the *intent* is wrong. Most detectors implicitly assume compromised credentials; this one has none.

7 · Pillar 2 — Generate: simulation and fidelity evidence

The feasible-action constraint

The standing criticism of adversarial machine learning on tabular data is that papers perturb features an attacker cannot actually control. That produces impressive numbers and unusable systems. Aegis's generators may move **only** what a real attacker controls.

LEVERS THE SIMULATOR MAY MOVE

- Amount, and how it splits across attempts
- Timing, inter-arrival cadence, burst shape
- Merchant and MCC selection
- Device and channel presentation
- Sequencing — probe, escalate, cash out
- Text in merchant-controlled fields

HELD INVARIANT — NOT THE ATTACKER'S TO CHANGE

- The victim's own historical baseline
- Issuer-side risk state and scoring
- Network-assigned identifiers and tokens
- AVS / CVV results returned by the issuer
- Another cardholder's genuine behaviour
- Any label the defence later assigns

This costs headline numbers — it is far easier to score well against attacks that cheat by editing issuer-side fields. But an attack requiring a rewrite of the victim's own history is data corruption, not an attack. The constraint is what makes the detection results mean anything operationally.

Fidelity evidence — measured, not asserted

The brief judges fidelity *instrumentally*: the data must be "genuinely useful for training and stress-testing a defense". So `fidelity.py` measures it, and the results land in `metrics.json`. 9/9 marginals within published reference bands; max raw-field univariate auc 0.8958, mean attack/legit overlap 0.6557.

Part 1 — generated marginals against published reference bands

Measure	Value	Reference band	In band
legit baseline fraud bps	0.0	0.0 – 60.0	✓
cnp share	0.42365	0.3 – 0.6	✓
cross border share	0.10309	0.03 – 0.2	✓
night share	0.03422	0.02 – 0.18	✓
benford mad	0.00101	0.0 – 0.015	✓
log amount abs skew	0.00471	0.0 – 1.0	✓
dispersion index	35.22185	1.0 – 400.0	✓
mcc gini	0.44472	0.25 – 0.85	✓
mean primary device share	0.97024	0.55 – 1.0	✓

Bands come from public sources — PSD2 RTS Annex fraud-rate bands, and Nigrini's published thresholds for Benford conformity — and are deliberately wide: they are sanity bands for a synthetic corpus, not calibration targets to overfit.

Benford's law. Real transaction amounts follow Benford's distribution of leading digits, a standard forensic-accounting test. Ours has a mean absolute deviation of **0.0010**, inside Nigrini's "close conformity" threshold of 0.006 — and we did not tune for it.

Part 2 — non-separability (the anti-"trivially separable" evidence)

If attack traffic came from an obviously different process, any classifier would score ~ 1.0 and the entire evaluation would be meaningless. So we measure univariate AUC on **raw** authorization fields, with no engineered features. Here, **low is good**.

Raw field	Univariate AUC	Attack/legit overlap
cross_border	0.896	0.208
card_present	0.788	0.424
merchant_age_days	0.706	0.550
hour_of_day	0.651	0.667
amount	0.559	0.860
network_token_used	0.552	0.895
is_recurring	0.507	0.986

Max raw-field AUC **0.8958**; mean attack/legit overlap **0.6557**. The `amount` row is the important one: at AUC 0.559 the generator plainly does **not** take the usual shortcut of making fraud large — the failure mode that renders most synthetic fraud corpora trivially separable.

`cross_border` is the highest single field, and that is realistic rather than an artefact: cross-border genuinely carries materially elevated fraud rates in live portfolios. If our cross-border fraud rate matched domestic, *that* would be the fidelity failure.

The most camouflaged vectors — highest overlap with legitimate traffic — are `VELOCITY_EVASION` 0.691, `GENAI_DOC_FARM` 0.640, `TRA_THRESHOLD_GAMING` 0.636, `AGENT_IMPERSONATION` 0.626. These are the same vectors the detector performs worst on, so the fidelity measurement and the detection results corroborate each other independently.

Safety

Network isolation by construction. The simulator has no network client at all. This is enforced by an **AST-level test** that parses every simulator module and fails the build if a networking import, `subprocess`, or dynamic execution appears. Competition Rules §3(b) require that adversarial testing "does not target live systems, payment infrastructure or third parties" — so this is a rules requirement met in code, not a promise in prose.

8 · Pillar 3 — Defend: features, cascade, explainability

The feature layer — 57 strictly causal features

Family	Examples	What it captures
Velocity	transactions in the last hour / day; amount velocity; distinct merchants per window	Bursts and automation
Own-baseline deviation	amount z-score against this account's own history; new-merchant flag; unusual hour for <i>this</i> cardholder	Change relative to the individual, not to the population — a ₹5,000 purchase is normal for one person and alarming for another
Graph	cards per device; devices per card; ring component size; fan-in	Shared infrastructure, invisible per transaction
Verification coherence	AVS/CVV/3DS agreement; exemption claimed on a high-risk payment; token absent where expected	Combinations that should not co-occur

"**Strictly causal**" means every feature is computable from information available at the moment of authorization. No feature sees the label, and none sees the future. The test suite proves it: features recomputed on a truncated prefix of history must be *identical* to the full-history values for those rows.

The three-stage cascade

```
transaction
|
[1] 39 deterministic rule signals      ← fast, cheap, runs on 100% of traffic
|
[2] HistGradientBoostingClassifier    ← learned patterns
|
[3] graph structure                  ← expensive; runs on the riskiest 20% only
|
[arbiter] combines the three additively
|
[isotonic calibration] → final risk score in [0,1] that means what it says
```

The airport-security analogy: a metal detector everyone passes through, an X-ray for bags, and a full manual search reserved for the few. Stage 3 is gated by an explicit **compute budget** (the riskiest 20%) rather than a score threshold, because a threshold allows cost to spike exactly when an attack floods the high-risk band. A budget cannot. That is how p99 stays at 18.8 ms.

Explainability — exact by construction

A declined payment may legally require a reason, and model governance expects decisions to be reconstructable. Every alert carries four things:

1. **The decision** — calibrated risk, band, recommended action (allow / review / step up / block), alongside the payment's own verification attributes.

- 2. **An exact score decomposition** — additive contributions to the arbiter's log-odds. "Log-odds" simply means a scale on which evidence adds up. These are not estimates: the terms sum to the score, and the test suite verifies the reconciliation against the model's own decision function.
- 3. **Reason codes** — ranked, in analyst language rather than feature names (e.g. *"Unusual burst of transactions within the hour"*, weight 0.80).
- 4. **A counterfactual** — what would have to change for the payment to score benign (e.g. *"would likely fall below review absent the velocity_burst signal"*).

The honest boundary. Per-row attribution *inside* the gradient-boosted component is **not** claimed. Its contribution appears as a single exact term, and its feature importance is reported globally and labelled as global. SHAP could not be installed (`numba` fails on Python 3.14), and shipping an approximate explainer while calling it attribution would be worse than naming the limit — the literature shows fluent rationales raise analyst confidence without raising accuracy.

Signals deliberately not implemented

All 39 of 39 distinct declared signals are implemented. These are published as out of scope, each with its reason:

Signal	Why not
<code>session_duress_pattern</code>	requires session/interaction telemetry, outside the auth schema
<code>refund_ratio_anomaly</code>	requires credit/refund messages, outside the auth schema
<code>post_delivery_dispute</code>	requires dispute lifecycle data, outside the auth schema
<code>repeat_claimant_pattern</code>	requires dispute lifecycle data, outside the auth schema
<code>synchronised_timing</code>	covered in practice by <code>ring_component</code> + <code>machine_cadence</code>
<code>graph_fanin</code>	emitted by the graph stage, not the rule stage
<code>ring_component</code>	emitted by the graph stage, not the rule stage
<code>injection_pattern_in_text</code>	emitted by the text-safety stage

9 · Evaluation — methodology and full results

How the split is done, and why it matters most

temporal with delay block — train 58,667, test 27,078, with 5% of the timeline **discarded between them**. Test-set fraud prevalence is 2.98%.

Chargeback labels arrive weeks after the transaction. A random train/test split would let the model learn from payments that happen *after* the ones it is tested on — the future leaking into the past, producing a score that cannot be reproduced in production. Discarding a slice of the timeline simulates that reporting lag honestly.

Headline results

PR-AUC
0.9435 95% CI 0.931 – 0.957, bootstrapped

ROC-AUC
0.9894 shown for comparability; optimistic under imbalance

DECISION LATENCY
18.79 ms p99 · p50 13.65 ms · p95 16.23 ms

ZERO-DAY RECALL
0.718 6 vectors never trained on

Why PR-AUC is the headline: at 2.98% prevalence a model that blocks nothing scores 97.0% accuracy. Accuracy is meaningless here. ROC-AUC is optimistic because the true-negative pool is enormous. PR-AUC concentrates on the positive class, which is the one that matters.

Three operating points — one detector, three settings of the dial

Operating point	Threshold	Precision	Recall	Notes
Best-F1	0.798	0.972	0.891	F1 0.929; FPR 0.00080
Capacity-constrained 1% analyst review budget	1.000	0.993	0.334	271 alerts; ceiling 0.336
Prevalence-matched budget sized to actual fraud rate	0.526	0.909	0.909	806 alerts; not budget-capped

Confusion matrix at the best-F1 point: **TP 718** fraud correctly caught · **FP 21** honest payments wrongly flagged · **FN 88** fraud missed · **TN 26,251** honest payments correctly approved.

Read the ceiling before reading the recall. Recall here is BOUNDED BY THE BUDGET, not by the model: with a 1% alert budget and 2.98% test prevalence, no detector could exceed the ceiling shown. Read this number as 'value captured per unit of analyst effort', not as model recall. At a 1% alert budget and 2.98% prevalence the maximum recall *any* detector could achieve is **0.336**. We reach 0.334 — 99.3% of the mathematical ceiling. If a hospital has 10 beds and 30 patients, treating 10 is a bed shortage, not a doctor failure.

Money and customer impact

Value detection rate **0.941** — the share of attempted fraud *value* stopped, not just count

Insult rate 0.00080 — share of legitimate payments declined at the best-F1 point

Note on absolute values Amounts are summed across a multi-currency synthetic population, so the absolute totals carry no single currency unit. Read the ratio, not the totals.

Calibration — does 0.9 really mean 90%?

Brier 0.00445 · Expected Calibration Error (10-bin) **0.00378** · method: isotonic regression on a held-out temporal slice.

A calibrated score behaves like an honest weather forecaster: on days they say "70% chance of rain", it rains about 70% of the time. This matters because a bank sets policy on expected loss — if 0.9 does not mean 90%, a cost-based block threshold cannot be justified.

Zero-day generalisation — the novelty evidence

6 vectors removed from training **entirely**, then scored at a threshold calibrated on **seen traffic only** — no retuning on the held-out data. Aggregate: **0.718** across 975 transactions.

Held-out vector	n	Recall	Mean risk	Hard
BIN_ENUMERATION_BURST	300	1.000	1.000	—
SYNTH_ID_BUSTOUT	168	0.982	0.999	—
ATO_CREDENTIAL_STUFF	117	0.974	0.997	—
APP_SCAM_LLM	30	0.800	0.988	yes
ROMANCE_PIG_BUTCHERING	144	0.500	0.931	yes
AGENT_IMPERSONATION	216	0.116	0.866	—

Recall on attacks the model trained on measures memorisation. This measures whether a causal feature layer transfers to fraud that *did not exist* when the model was fit.

Per-attack recall — worst first

At the capacity-constrained operating point, sorted so the failures appear first rather than buried at the bottom.

Vector	Category	n	Recall @1%	Mean risk	Hard	Sev
REFUND_ABUSE_COLLUSION	Merchant fraud	2	0.000	0.846	—	3
ADAPTIVE_MIMICRY	Adaptive evasion	68	0.015	0.224	yes	5
SIM_SWAP_OTP	Account takeover	24	0.042	0.272	yes	5
DEEPFAKE_KYC_ONBOARD	Deepfake / KYC	40	0.825	0.984	—	5
TRA_THRESHOLD_GAMING	Adaptive evasion	48	0.854	0.995	yes	4
VOICE_CLONE_ATO	Account takeover	28	0.893	0.996	—	5
APP_SCAM_LLM	Scam / social engineering	10	0.900	0.985	yes	5
SCA_EXEMPTION_ABUSE	Adaptive evasion	110	0.909	0.982	yes	4
VELOCITY_EVASION	Adaptive evasion	72	0.931	0.998	yes	4
GENAI_DOC_FARM	Synthetic identity	40	0.950	0.992	—	4
ATO_CREDENTIAL_STUFF	Account takeover	38	1.000	1.000	—	4
FAKE_STOREFRONT	Merchant fraud	23	1.000	1.000	—	4
ROMANCE_PIG_BUTCHERING	Scam / social engineering	48	1.000	1.000	yes	5
INVOICE_REDIRECT_BEC	Scam / social engineering	4	1.000	1.000	yes	4
CARD_TESTING_MICRO	Enumeration	80	1.000	1.000	—	3
BIN_ENUMERATION_BURST	Enumeration	100	1.000	1.000	—	3
MULE_FANOUT	Fraud ring	31	1.000	1.000	—	5
AGENT_PROMPT_INJECTION	Agentic commerce	8	1.000	1.000	—	4
MANDATE_REPLAY_ABUSE	Agentic commerce	32	1.000	1.000	yes	5

10 · Where it fails — stated plainly

The one genuine model failure

ADAPTIVE_MIMICRY — recall 0.015 at the 1% budget, and crucially a mean risk of only 0.224. The low mean risk is what makes this a real miss rather than a queue-capacity artefact: the model genuinely does not find it suspicious. The attack learns the victim's own baseline and stays inside it, which defeats deviation-based features *by construction*.

It is corroborated independently by the fidelity measurement, where mimicry is among the vectors overlapping legitimate traffic most. **This is the row we would fix first.** The likely approach is a "baseline over-conformity" feature: a replayed baseline is *less* noisy than genuine human behaviour, so anomalously low variance is itself a signal.

Second weakest

AGENT_IMPERSONATION — zero-day recall 0.116. In an authorization message, a legitimate agentic purchase and an impersonated one are nearly indistinguishable. Separating them requires agent-identity attestation that the schema does not yet carry. This is a roadmap item, not a tuning problem.

Rows that look worse than they are

REFUND_ABUSE_COLLUSION shows 0.000 recall on **n = 2** transactions in this split — that is noise, not a measurement. Its mean risk is 0.846, so the model did rank it as risky; it simply lost the competition for 271 review slots.

Prevalence caveat

Synthetic fraud prevalence here is 3.83%, deliberately higher than the ~0.1-1% seen in live card portfolios (PSD2 RTS Annex reference bands are 1-13 bps; Stripe reports ~1 in 1,000). A higher rate is required to train and evaluate 25 distinct attack vectors on a synthetic corpus. Threshold-dependent metrics (precision, alert-rate recall, insult rate) are prevalence-sensitive and would shift in a live portfolio; PR-AUC, calibration and latency are the more transferable figures.

Explainability boundary

Restated because it matters: no per-row attribution is claimed inside the gradient-boosted component. Its importance is global and labelled as global.

11 · The dashboard — all seven panels

The header carries two compliance badges — **SYNTHETIC DATA** and **NETWORK-ISOLATED** — plus a live/offline state. The navigation is grouped into **red team** and **blue team** so both halves of the brief are visible.

Panel	Side	What it shows
Overview	—	Transactions in scope, attack traffic injected, high/critical alerts, vectors available; alert-band distribution; the four verified headline metrics; the closed-loop diagram; recent campaigns with detection rates
Attack Simulator	Red	The library of 25 vectors filtered by category. Selecting one shows its mechanics, what GenAI changed, its MITRE ATLAS mapping, its expected detection signals declared up front , and a strength slider from "subtle · fewer entities" to "aggressive · at scale". Launch campaign injects it live.
Live Stream	Blue	The scored authorization feed — risk, band, recommended action. Filter to high & critical to see the real analyst queue.
Investigate	Blue	Alert queue plus, for the selected payment: the decision and its verification attributes, the exact additive score decomposition, ranked reason codes, and the counterfactual.
Fraud Network	Blue	The subgraph induced by <i>shared</i> infrastructure only — entities touching two or more distinct cards. Unpruned it is mostly singleton pairs and the structure disappears.
Performance	Blue	The full evaluation: split description, headline metrics, three operating points, calibration curve, zero-day table, per-attack recall worst-first, signal coverage, and the stated limitations.
Audit Trail	—	Append-only log of every environment change, campaign and analyst action. Append-only means entries can be added but never edited or deleted.

Alert bands

Band	Meaning	Action
LOW	Normal	Approve silently
MEDIUM	Slightly unusual	Monitor
HIGH	Suspicious	Human review
CRITICAL	Almost certainly fraud	Block

LOW is charted as a bar but excluded from the scaled chart deliberately: it holds tens of thousands of rows and would flatten the three actionable bands into invisibility. Colour is always paired with an explicit text label, never used alone, for accessibility.

12 · How to run it

First time only

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
cd frontend && npm install && cd ..
.venv/bin/python -m backend.app.evaluate      # regenerates all metrics (~3 min)
```

Every session — two terminal windows

```
# WINDOW 1 — backend
.venv/bin/uvicorn backend.app.api:app --port 8000

# WINDOW 2 — dashboard
cd frontend && npm run dev
```

Then open **<http://localhost:5173>** and click **Start environment**. The build takes 10–15 seconds; wait for the header badge to read **ENVIRONMENT LIVE** before presenting.

Windows 1 and 2 stay occupied — those processes run until stopped with `Ctrl+C`. Use a **third** window for any other command.

Health checks

```
curl -s localhost:8000/api/health      # -> {"status":"ok", ...}
curl -s localhost:8000/api/health | grep -o '"ready":[a-z]*'  # false = still building
```

13 · How to test it

```
./scripts/verify.sh --full      # everything; must end "VERIFIED – safe to push"
```

Stage	What it proves
6 module self-checks	Each module works standalone
113 pytest tests	Nothing regressed
Secrets & compliance scan	No credentials, no competition captures, no build artifacts tracked
Metrics freshness guard	Fails if any backend source is <i>newer</i> than <code>metrics.json</code> — stale metrics silently poison every published figure
Docs-agree guard	Fails if any document quotes a PR-AUC or p99 that disagrees with the artifact
TypeScript typecheck + Vite build	Frontend compiles cleanly
Playwright browser smoke test	The real demo path works end to end

Start both servers before testing. Without them the browser smoke test *skips* rather than fails — and prints green. A skipped check that looks like a pass is more dangerous than a red failure.

Narrower commands

```
.venv/bin/python -m pytest backend/tests -q           # 113 tests, ~30s
.venv/bin/python -m pytest backend/tests/test_security.py -q # 21 compliance tests
.venv/bin/python -m backend.app.fidelity              # fidelity self-check
.venv/bin/python -m backend.app.evaluate              # regenerate metrics
.venv/bin/python scripts/make_deck.py                 # rebuild the deck
python3 scripts/make_pdf.py                           # rebuild this document (system python)
cd frontend && npx tsc --noEmit                         # typecheck only
```

What the four test suites protect

Suite	Guards against
Data pipeline	Schema drift, duplicate transaction ids, label leakage
Detection	Cascade regression, calibration drift, explainer/model disagreement
Security (21 tests)	Network egress in the simulator (AST-level), PAN-like or Luhn-valid identifiers, personal names, untruncated IPs, committed secrets, non-permissive licences, operational fraud instructions in the taxonomy
API	Endpoint contracts, WebSocket lifecycle, append-only audit behaviour

14 · Glossary — every term used

Payments

Term	Meaning
Authorization	The real-time request asking the issuer to approve a payment. Aegis scores this message.
PAN	Primary Account Number — the long number on a card. Never present in this system ; tokens are used throughout.
Network token	A substitute value standing in for the PAN, so the real number never circulates.
MCC	Merchant Category Code — a 4-digit industry code (5999 = miscellaneous retail).
ECOM / POS / MOTO	Online / physical card machine / mail-or-telephone order.
Card-not-present (CNP)	The physical card was not used — typically online. Structurally higher risk.
AVS	Address Verification Service — did the billing address match?
CVV	The 3-digit security code.
3-D Secure	The extra bank authentication step (Mastercard Identity Check, Verified by Visa).
SCA	Strong Customer Authentication — EU PSD2 requirement for multi-factor authentication.
SCA exemption	A rule permitting the check to be skipped for low-risk payments. Attackers game the bands.
TRA	Transaction Risk Analysis — an exemption route based on the acquirer's fraud rate.
PSD2 / RTS	EU payment services regulation and its Regulatory Technical Standards, which publish reference fraud-rate bands.
ISO 8583	The international standard for card authorization message formats.
Chargeback	A customer dispute reversing a payment — the mechanism that produces fraud labels, weeks late.
APP fraud	Authorised Push Payment fraud — the victim is tricked into authorising it themselves.
Pig-butcher	A long-con romance-plus-investment scam.
Mule account	An account used to receive and forward criminal proceeds.
BIN enumeration	Automated guessing of valid card numbers by testing small charges.
Bust-out	Building good credit history, then maxing everything out and disappearing.
Transaction laundering	Processing payments for one business under another's merchant account.
Insult rate	How often a legitimate customer is wrongly declined. Industry term.
Step up	Do not decline — request additional proof, such as an OTP.
Issuer / Acquirer	The cardholder's bank / the merchant's bank.
KYC	Know Your Customer — identity verification at account opening.

Machine learning and statistics

Term	Meaning
True positive (TP)	Fraud, correctly caught.
False positive (FP)	Legitimate payment, wrongly flagged — an angry customer.
False negative (FN)	Fraud, missed — money lost.
True negative (TN)	Legitimate payment, correctly approved.
Precision	Of everything flagged, the share that really was fraud. "Am I crying wolf?"
Recall	Of all real fraud, the share caught. "Am I missing thieves?"
F1	The harmonic mean of precision and recall — a single balanced score.
PR-AUC	Area under the precision–recall curve: overall quality across every threshold. The honest headline under class imbalance.
ROC-AUC	Area under the receiver-operating-characteristic curve. Optimistic when positives are rare, because the true-negative pool is huge.
Class imbalance	One class vastly outnumbers the other. Here fraud is ~3% of rows.
Prevalence	The share of rows that are fraud.
Calibration	Whether a predicted probability matches observed frequency — does 0.9 mean 90%?
ECE	Expected Calibration Error — average gap between predicted and observed, across bins.
Brier score	Mean squared error of probabilistic predictions. Lower is better.
Isotonic regression	A monotonic, non-parametric method for fixing miscalibrated scores.
Log-odds	A scale on which independent pieces of evidence can be <i>added</i> . Makes the explanation additive.
Gradient boosting	An ensemble of small decision trees, each correcting its predecessors' errors.
Temporal split	Train on earlier data, test on later data — mimics production.
Delay block	A discarded slice of timeline between train and test, reflecting late label arrival.
Label leakage	When a feature encodes the answer, inflating scores unreproducibly.
Zero-day	An attack type absent from training entirely.
Bootstrapped CI	A confidence interval obtained by resampling, rather than assuming a distribution.
p50 / p95 / p99	Median / 95th / 99th percentile latency. p99 is the worst-case that matters.
Univariate AUC	Discriminative power of a <i>single</i> feature. Used here to prove no raw field gives the attacks away.
Overlap coefficient	How much two distributions share, 0–1. High overlap between attack and legitimate traffic means realistic camouflage.
Benford's law	The distribution of leading digits in natural numeric data. A standard forensic test.

MAD	Mean absolute deviation — used here against Benford's expected frequencies.
Gini coefficient	A concentration measure, used here for how unevenly volume spreads across MCCs.
Index of dispersion	Variance divided by mean. Above 1 means bursty rather than uniformly random.
MITRE ATLAS	A public catalogue of adversarial techniques against AI systems.
Counterfactual	The minimal change that would flip the decision.
AST	Abstract Syntax Tree — the parsed structure of source code. Used to prove the simulator cannot reach a network.

15 · Competition compliance

Requirement	Source	How it is met
Synthetic, anonymised or authorised sample data only; no real cardholder, PII or production payment data	Rules §3(a)	100% self-generated. No PAN exists. Every row carries <code>synthetic: true</code> . Enforced by tests for PAN-like patterns, Luhn validity, personal names and untruncated IPs.
Adversarial testing must not target live systems, payment infrastructure or third parties	Rules §3(b)	The simulator has no network client. An AST-level test fails the build if a networking import, subprocess or dynamic execution appears in any simulator module.
Responsible AI, cybersecurity and disclosure practices	Rules §3(c)	<code>docs/security.md</code> ; a test asserts the taxonomy contains no operational fraud instructions — it describes behaviour to detect, not recipes to reproduce.
Open-source dependencies must be OSI-approved and must not limit commercial use	Kaggle Foundational §6(c)	All 41 Python and 121 npm packages audited; MIT / BSD / Apache-2.0 / ISC / MPL-2.0 only. Zero AGPL, GPL or SSPL. Inventory in <code>requirements.txt</code> .
No private sharing of competition code during the competition period	Kaggle Foundational §6(a)	Repository is private, verified by an unauthenticated API request returning 404. Published after judging concludes.
A valid submission must include all three artifacts	Rules §2, §3	Code repository · walkthrough deck (<code>.pptx</code> , 15 slides) · working web prototype.
Draft work is not considered	Rules §2	Action required: the writeup must be actively <i>submitted</i> in the Kaggle Writeups section, not left in draft.

16 · Repository map

```
research/          verified competition rules, threat landscape, existing solutions, sources
docs/
  architecture.md   system design
  decisions.md       every significant choice and its rationale
  threat-model.md   attacker capabilities and assumptions
  fraud-taxonomy.md all 25 vectors in detail (generated)
  detection-methodology.md features, cascade, calibration
  evaluation.md      full results (generated)
  security.md        responsible AI and security posture
  demo-flow.md       the 7-minute live demo script
  deployment.md      production considerations
  presenter-guide.md run/test, pitch at 3 lengths, judge Q&A bank
  submission-writeup.md the Kaggle Writeups text
  current-state.md   compact project state
backend/app/        schema · generator · attacks · features · fidelity · detect · explain ·
                    evaluate · api
backend/tests/      113 tests across 4 suites
frontend/src/       App.tsx + 7 panels + shared components
scripts/            verify.sh · make_deck.py · make_pdf.py · gen_docs.py · ui_smoke.py
artifacts/          metrics.json · aegis-walkthrough.pptx · aegis-project-explained.pdf ·
                    screenshots/ · audit.db
```

Reproduce everything from a clean checkout

```
git clone <repo> && cd ThreatForge
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
cd frontend && npm install && cd ..
.venv/bin/python -m backend.app.evaluate
./scripts/verify.sh --full
```

Generated by `scripts/make_pdf.py` from `artifacts/metrics.json` (evaluation run 2026-08-17T19:03:48Z). Every figure in this document is read from that artifact; none is typed by hand.